

1 Write a Python program demonstrating CRUD operations on a dictionary.

Aim

To write a Python program that performs CRUD operations (Create, Read, Update, Delete) on a dictionary using a menu-driven approach.

Algorithm

1. Initialize an empty dictionary.
2. Display a menu with options:
 1. Create (add a key-value pair)
 2. Read (display dictionary contents)
 3. Update (modify value for a given key)
 4. Delete (remove a key-value pair)
 5. Exit
3. Accept user choice.
4. Perform the corresponding dictionary operation.
5. Repeat until the user chooses to exit.

CODE

```
# CRUD operations on a dictionary
```

```
def dictionary_crud():  
    my_dict = {} # Step 1: Initialize empty dictionary  
  
    while True:  
        print("\n--- Dictionary CRUD Menu ---")  
        print("1. Create (Add key-value pair)")  
        print("2. Read (Display dictionary)")  
        print("3. Update (Modify value)")
```

```
print("4. Delete (Remove key)")
```

```
print("5. Exit")
```

```
choice = int(input("Enter your choice (1-5): "))
```

```
if choice == 1: # Create
```

```
    key = input("Enter key: ")
```

```
    value = input("Enter value: ")
```

```
    my_dict[key] = value
```

```
    print(f"Added: {key} → {value}")
```

```
elif choice == 2: # Read
```

```
    print("Current Dictionary:", my_dict)
```

```
elif choice == 3: # Update
```

```
    key = input("Enter key to update: ")
```

```
    if key in my_dict:
```

```
        value = input("Enter new value: ")
```

```
        my_dict[key] = value
```

```
        print(f"Updated: {key} → {value}")
```

```
    else:
```

```
        print("Key not found!")
```

```
elif choice == 4: # Delete
```

```
    key = input("Enter key to delete: ")
```

```
    if key in my_dict:
```

```
        del my_dict[key]
```

```
        print(f"Deleted key: {key}")
```

```
    else:
```

```
        print("Key not found!")

    elif choice == 5: # Exit
        print("Exiting program. Goodbye!")
        break

    else:
        print("Invalid choice! Please enter between 1-5.")

# Run the program
dictionary_crud()
```

OUTPUT

--- Dictionary CRUD Menu ---

1. Create (Add key-value pair)
2. Read (Display dictionary)
3. Update (Modify value)
4. Delete (Remove key)
5. Exit

Enter your choice (1-5): 1

Enter key: name

Enter value: Yogin

Added: name → Yogin

Enter your choice (1-5): 2

Current Dictionary: {'name': 'Yogin'}

Enter your choice (1-5): 3

Enter key to update: name

Enter new value: Alex

Updated: name → Alex

Enter your choice (1-5): 4

Enter key to delete: name

Deleted key: name

Enter your choice (1-5): 5

Exiting program. Goodbye!

Conclusion

- This program demonstrates CRUD operations on a dictionary.
- Users can add, view, update, and delete key-value pairs interactively.
- It shows how dictionaries can be manipulated dynamically in Python.

2 Write a program to convert a list to a tuple and vice versa.

Aim

To write a Python program that demonstrates conversion between a list and a tuple using built-in functions.

Algorithm

1. Create a sample list.
2. Convert the list into a tuple using .
3. Create a sample tuple.
4. Convert the tuple into a list using .
5. Print results to verify conversions.

CODE

```
# Program to convert list to tuple and vice versa
```

Step 1: Convert list to tuple

```
my_list = [10, 20, 30, 40]
```

```
print("Original List:", my_list)
```

```
my_tuple = tuple(my_list)
```

```
print("Converted Tuple:", my_tuple)
```

Step 2: Convert tuple to list

```
my_tuple2 = (100, 200, 300, 400)
```

```
print("\nOriginal Tuple:", my_tuple2)
```

```
my_list2 = list(my_tuple2)
```

```
print("Converted List:", my_list2)
```

OUTPUT

Original List: [10, 20, 30, 40]

Converted Tuple: (10, 20, 30, 40)

Original Tuple: (100, 200, 300, 400)

Converted List: [100, 200, 300, 400]

Conclusion

- Python provides built-in functions and for easy conversion.
- Lists are mutable (can be changed), while tuples are immutable (cannot be changed).
- This program demonstrates how to switch between these two data structures seamlessly.